

Python for Web Development

Lesson 8: Testing and Debugging

Testing and Debugging - Study Notes

Key Concepts

1. **Unit testing with pytest** – writing isolated test functions, using fixtures, parametrization, and assertions.
2. **Flask test client** – simulating HTTP requests to a Flask app without running a server, checking responses and context.
3. **Debugging tools** – built in ``pdb``, Flask's interactive debugger, and IDE breakpoints for step by step inspection.
4. **Test organization & coverage** – structuring test modules, using ``conftest.py``, and measuring coverage with ``pytest-cov``.
5. **Continuous testing** – running tests automatically with ``tox`/`nox`` or CI pipelines to catch regressions early.

Summary

Testing and debugging are essential practices for delivering reliable Python and Flask applications.

Pytest has become the de facto standard for unit testing because of its simple syntax, powerful fixture system, and rich plugin ecosystem. By writing small, focused test functions that assert expected behavior, developers can verify individual units of code and detect regressions quickly.

When the code under test is a Flask web application, the **Flask test client** provides a lightweight way to exercise routes, view functions, and request handling without launching a real server. The client mimics a browser, allowing you to send GET, POST, PUT, etc., inspect response data, status codes, headers, and even the Flask ``g`` or ``session`` objects. Combined with pytest fixtures that create a fresh app and client per test, you get fast, deterministic integration tests.

Debugging complements testing by helping you understand *why* a test fails. Python's built in ``pdb`` module, the interactive debugger that ships with Flask (enabled in development mode), and modern IDE breakpoints all let you pause execution, inspect variables, and step through code line by line. Using these tools effectively reduces time spent chasing bugs and improves confidence in the correctness of your code.

Important Points

- **pytest basics**
 - Test files must be named ``test_*.py`` or ``*_test.py``.
 - Test functions start with ``test_``.
 - Use plain ``assert`` statements; pytest rewrites them for rich introspection.
- **Fixtures**
 - Defined with ``@pytest.fixture``.

- Scope options: ``function`` (default), ``module``, ``class``, ``session``.
- Can be parametrized with ``@pytest.fixture(params=[...])`` or ``@pytest.mark.parametrize``.
 - **Flask test client**
- Created via ``app.test_client()`` or a fixture that yields the client.
- Use ``client.get('/path')``, ``client.post('/path', json={...})``, etc.
- Access ``response.status_code``, ``response.get_json()``, ``response.data``.
- ``with app.test_request_context():`` lets you inspect ``url_for``, ``g``, ``session`` outside a request.
 - **Debugging**
- Insert ``import pdb; pdb.set_trace()`` or ``breakpoint()`` to start an interactive session.
- Flask's built in debugger shows a traceback with an interactive console when ``app.debug = True``.
- IDEs (VS Code, PyCharm) allow conditional breakpoints and watch expressions.
 - **Test organization**
- Keep tests in a separate ``tests/`` directory.
- Common fixtures go in ``tests/conftest.py`` so they're auto discovered.
- Use ``pytest -q`` for concise output, ``pytest -vv`` for verbose, and ``pytest --maxfail=1`` to stop after the first failure.
 - **Coverage**
- Install ``pytest-cov``.
- Run ``pytest --cov=your_package`` to see which lines are exercised.
- Aim for >80 % coverage, but prioritize meaningful tests over raw percentages.
 - **Continuous testing**
- ``tox`` can run the same test suite against multiple Python versions.
- CI services (GitHub Actions, GitLab CI) execute ``pytest`` on each push/PR, preventing broken code from merging.

Examples

1p ã Simple pytest unit test with a fixture

```
```python
```

**src/calculator.py**

```
def add(a, b):
 return a + b
def divide(a, b):
 if b == 0:
 raise ValueError("division by zero")
 return a / b
```

```
```
```

```
```python
```

**tests/test\_calculator.py**

```

import pytest
from src.calculator import add, divide
@pytest.fixture
def numbers():
 """Provide a pair of numbers for tests."""
 return (3, 5)
def test_add(numbers):
 a, b = numbers
 assert add(a, b) == 8
@pytest.mark.parametrize("a,b,expected", [
 (10, 2, 5),
 (9, 3, 3),
 (-4, 2, -2),
])
def test_divide_success(a, b, expected):
 assert divide(a, b) == expected
def test_divide_by_zero():
 with pytest.raises(ValueError, match="division by zero"):
 divide(1, 0)

```

\*Explanation\*:

- ``numbers`` fixture supplies reusable data.
- ``@pytest.mark.parametrize`` runs the same test with different inputs.
- ``pytest.raises`` asserts that an exception is raised with the expected message.

---

## 2b ã Flask route testing with the test client

```
```python
```

app/___init___.py

```

from flask import Flask, jsonify, request
def create_app():
    app = Flask(__name__)
    @app.route("/api/echo", methods=["POST"])
    def echo():
        payload = request.get_json()
        return jsonify({"you_sent": payload}), 201
    return app

```

```
```python
```

### tests/conftest.py

```

import pytest
from app import create_app
@pytest.fixture
def app():
 app = create_app()
 app.config.update({"TESTING": True})
 return app
@pytest.fixture
def client(app):
 return app.test_client()

```

```

```python

```

tests/test_api.py

```

def test_echo_endpoint(client):
    data = {"msg": "hello"}
    resp = client.post("/api/echo", json=data)
    assert resp.status_code == 201
    json_body = resp.get_json()
    assert json_body == {"you_sent": {"msg": "hello"}}

```

Explanation:

- `app` fixture creates a fresh Flask instance with `TESTING` flag, which disables error catching and enables the test client.
- `client` fixture yields `app.test\_client()`.
- The test sends a JSON payload, checks the status code, and validates the response JSON.

```

---
```

Quick Review

1. **What decorator marks a function as a pytest fixture, and how can you change its scope?**
2. **How do you send a JSON body with the Flask test client and retrieve the JSON response?**
3. **Name two ways to start an interactive debugging session in Python code.**
4. **What command runs pytest with coverage reporting for the package `myapp`?**
5. **Why is it useful to place common fixtures in `tests/conftest.py`?**

```

---
```

Further Reading

- **Advanced pytest** – custom markers, hooks, and plugin development.
- **Flask testing patterns** – testing blueprints, database fixtures, and authentication flows.
- **Debugging in VS Code / PyCharm** – configuring launch.json, watch expressions, and remote debugging.
- **Property based testing** – using `hypothesis` to generate many inputs automatically.

- **Continuous Integration** – setting up GitHub Actions workflows that run `pytest` and enforce coverage thresholds.

